



8



15



2250 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 2025/12/9

كلية الهندسة المعلوماتية

السنة الرابعة

Docker

عملي مشترك



نظم التشغيل

في المحاضرات السابقة، تعرّفنا على نظام Linux وتعلّمنا كيفية التعامل مع أوامره المختلفة، بالإضافة إلى كتابة واستخدام السكريبتات.

وفي هذه المحاضرة، سننتقل للحديث عن مفهومين غير مرتبطين مباشرة بنظام Linux ، لكنهما يُعدّان من المفاهيم الأساسية والمستخدمّة على نطاق واسع في الحياة العملية، وهما الافتراضية Virtualization و Containerization.

الافتراضية Virtualization

- الافتراضية هي عملية تشغيل نسخة افتراضية من نظام حاسوبي ضمن طبقة مجرّدة (Abstracted) عن العتاد الفيزيائي الحقيقي. وغالبًا ما يُقصد بها تشغيل عدة أنظمة تشغيل في الوقت نفسه على جهاز حاسوب واحد.
- بالنسبة للتطبيقات التي تعمل داخل النظام الافتراضي ، يبدو الأمر وكأنها تعمل على جهاز مستقل خاص بها ، حيث يكون نظام التشغيل والمكتبات و البرامج الأخرى فريدة للنظام الافتراضي Guest وغير مرتبطة بنظام المضيف (Host) الأساسي .
- كما توفّر الافتراضية مبدأ العزل (Isolation) ، حيث تبقى البرامج التي تعمل داخل آلة افتراضية معزولة وآمنة عن العمليات التي تجري داخل آلات افتراضية أخرى تعمل على نفس الجهاز المضيف.

■ أهمية الافتراضية :

بالنسبة لمسؤولي الخوادم، توفّر الافتراضية إمكانية تشغيل أنظمة تشغيل مختلفة على نفس الخادم، ولكن الأهم من ذلك أنها تتيح تقسيم النظام الكبير إلى عدة أنظمة أصغر، مما يسمح باستخدام موارد الخادم بكفاءة أعلى من قبل مستخدمين أو تطبيقات متعددة ذات متطلبات مختلفة، وبالتالي تقليل التكلفة وتوفير الموارد والمساحة.



Great things never come from comfort zones

■ ما هي الآلة الافتراضية (Virtual Machine) ؟

هي مورد حاسوبي يقوم بمحاكاة جهاز حاسوب فيزيائي باستخدام البرمجيات. تتيح تشغيل البرامج والتطبيقات داخل بيئة معزولة، وبشكل مستقل عن العتاد الحقيقي.

الفرق بين Guest و Host :

1. الجهاز المضيف (Host Machine):
هو الجهاز الفيزيائي الحقيقي الذي يوفر الموارد مثل المعالج، الذاكرة، التخزين، والشبكة.
2. جهاز الضيف (Guest Machine):
هو النظام الافتراضي الذي يعمل داخل الـ Hypervisor ، ويشغل نظام تشغيله وتطبيقاته الخاصة.

Hypervisor

ال Hypervisor ويُعرف أيضاً باسم (Virtual machine Monitor - VMM) هو برنامج مسؤول عن إنشاء وتشغيل الآلات الافتراضية (Virtual Machines)

■ وظيفة ال Hypervisor

يسمح لجهاز حاسوب واحد (Host) بدعم وتشغيل عدة آلات افتراضية (Guests) في الوقت نفسه، وذلك من خلال مشاركة موارد النظام مثل:

- المعالج (CPU)
- الذاكرة (Memory)
- التخزين (Storage)

■ فائدة ال Hypervisor

1. يوفر العزل (Isolation) بين الآلات الافتراضية
2. كفاءة في استخدام الموارد
3. مرونة عالية لمسؤولي الأنظمة والمطورين



الطريق لا يقل جمالاً عن الهدف

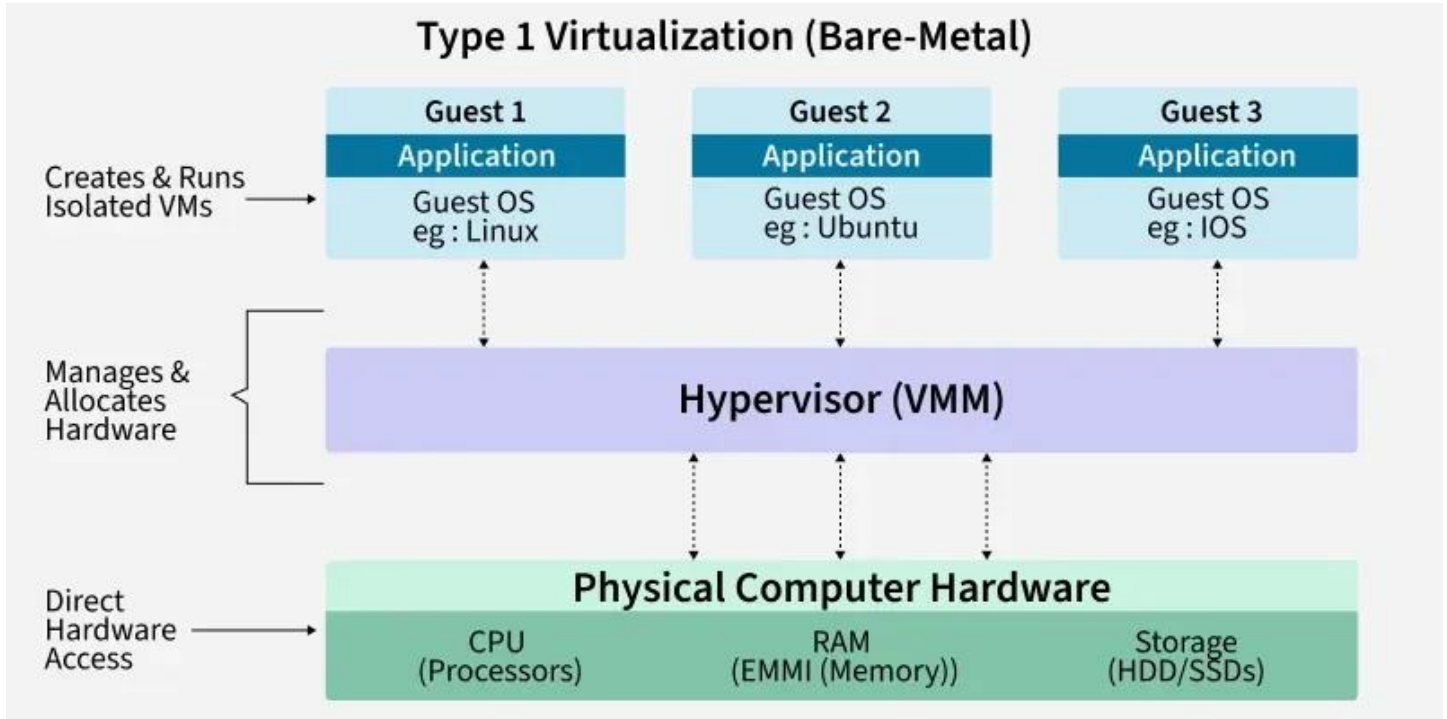
■ أنواع الـ Hypervisor

يوجد نوعان رئيسيان من الـ Hypervisor مستخدمان حالياً من قبل مسؤولي الأنظمة ومطوري البرمجيات:

1. النوع الأول (Type 1 – Bare Metal)

- يعمل مباشرةً على العتاد الفيزيائي للجهاز المضيف، ويحل محل نظام التشغيل.
- يُستخدم بشكل شائع في مراكز البيانات لما يقدمه من أداء عالٍ وقابلية توسع
- أمثلة:

- VMware vSphere / ESXi / ESX
- Microsoft Hyper-V
- Oracle VM Server
- Citrix Hypervisor

**توضيح البيئة في الصورة اعلاه :**

في البداية نلاحظ وجود الطبقة الفيزيائية، والتي تمثل العتاد الصلب للجهاز المضيف، وتشمل الموارد الأساسية مثل المعالج، الذاكرة، ووحدات التخزين.

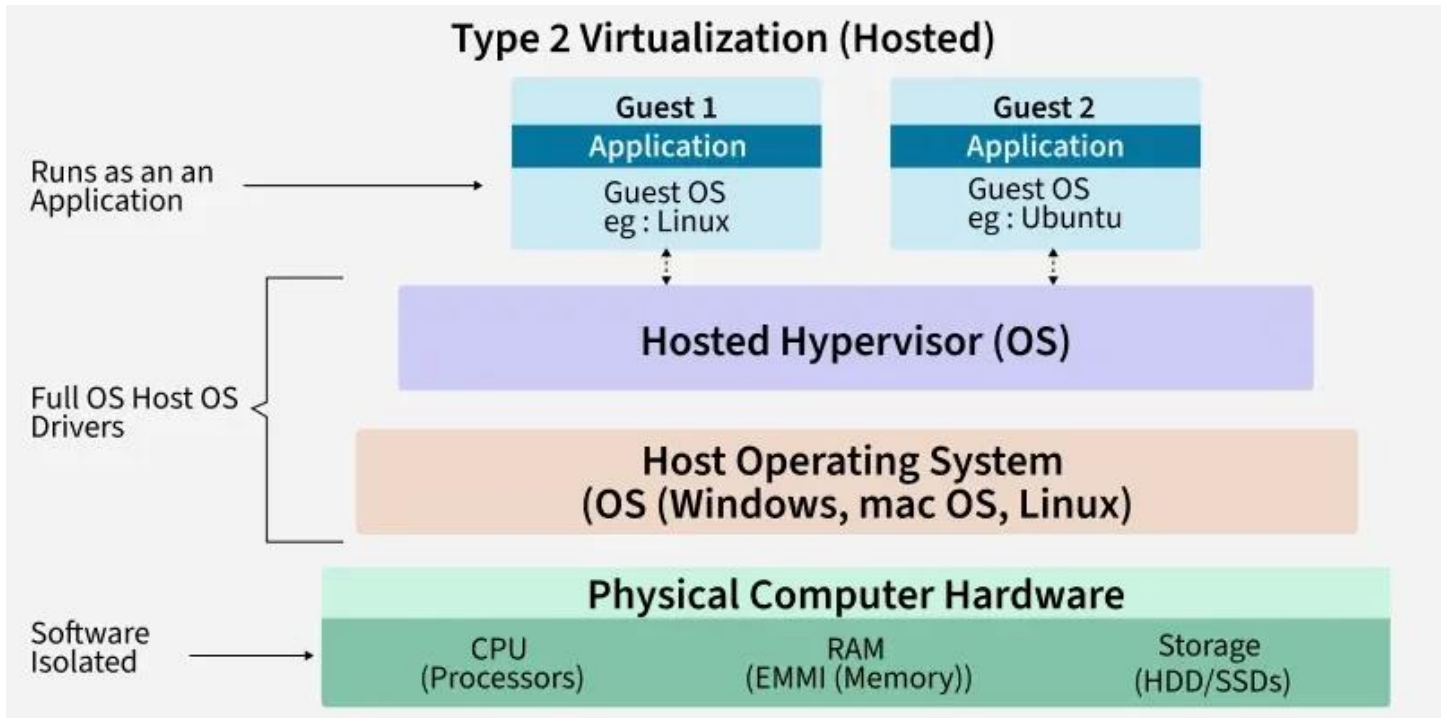
تعلوها مباشرة طبقة الـ Hypervisor ، والتي تتواصل بشكل مباشر مع العتاد الفيزيائي، وتتولى مسؤولية إدارة الموارد وتوزيعها ، إضافةً إلى التحكم في تشغيل الآلات الافتراضية وضمان عزلها عن بعضها البعض.

في أعلى البنية تأتي طبقة الآلات الافتراضية (Virtual Machines) ، حيث توجد ثلاث آلات افتراضية مستقلة ومعزولة، يعمل كل منها بنظام تشغيل خاص بها، وكأنها أجهزة منفصلة تشترك في نفس العتاد الفيزيائي.

2. النوع الثاني (Type 2 - Hosted)

- يعمل فوق نظام تشغيل موجود مسبقاً (مثل windows او linux)
- أسهل في التثبيت والاستخدام، لكنه أقل أداءً مقارنة بالنوع الأول
- يُستخدم عادةً في التطوير، الاختبار، والاستخدام الشخصي
- أمثلة:

- VMware Workstation
- VMware Fusion
- Oracle VirtualBox
- Oracle Solaris Zones



Containerization

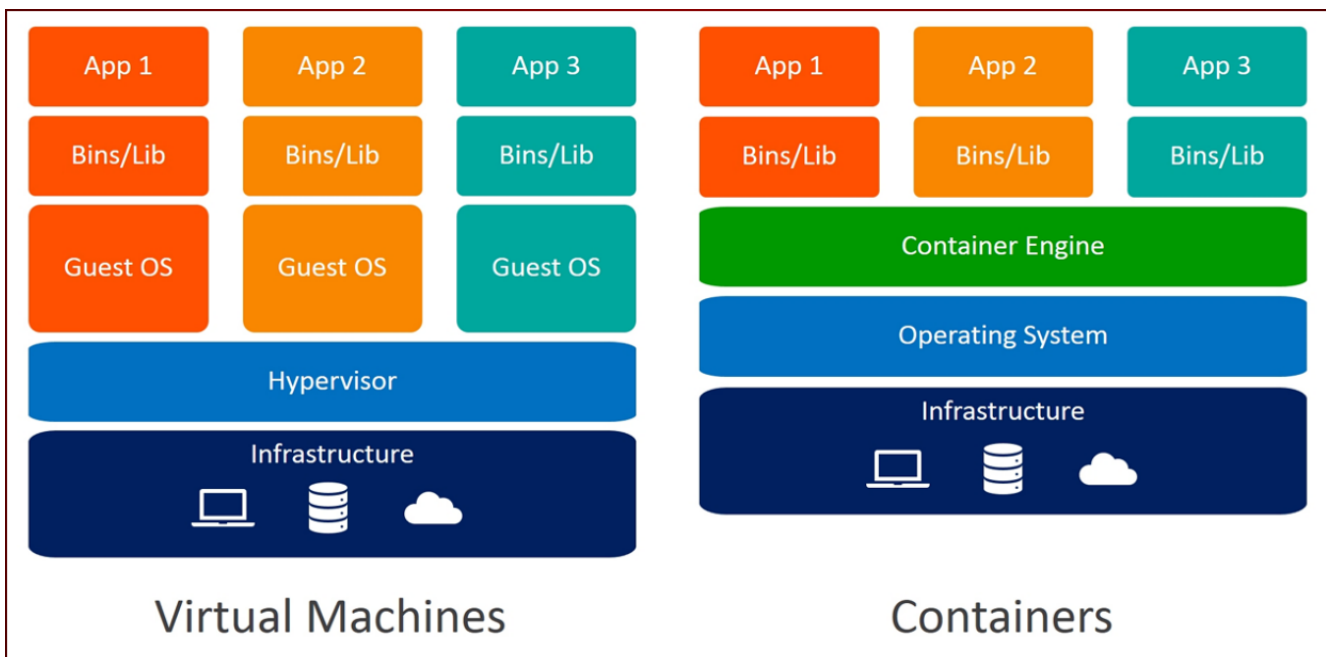
الآلات الافتراضية توفر عزلاً قوياً، لكنها قد تشكل تحدياً من ناحية الإدارة واستهلاك الموارد، خاصةً عند تشغيل عدد كبير منها على نفس الخادم. فكل آلة افتراضية تحتاج إلى نظام تشغيل كامل خاص بها، وتخصّص لها موارد محددة من الذاكرة والمعالج والتخزين، حتى وإن لم تكن هذه الموارد مستخدمة فعلياً طوال الوقت، مما يؤدي إلى هدر في موارد الجهاز المضيف.

بالإضافة إلى ذلك، فإن زمن إقلاع الآلات الافتراضية يُعد طويلاً نسبياً، نظراً لضرورة تشغيل نظام التشغيل بالكامل، الأمر الذي يؤثر على سرعة النشر والتوسع.

انطلاقاً من هذه التحديات، ظهرت تقنية الحاويات (Containerization)، حيث أنها:

1. بديل خفيف (Lightweight) عن الافتراضية التقليدية (Virtualization).
2. تعتمد على تغليف التطبيق داخل حاوية تحتوي على جميع متطلباته وبيئته التشغيلية.
3. بخلاف الآلات الافتراضية، لا تحتاج كل حاوية إلى نظام تشغيل مستقل، بل تشترك جميع الحاويات في نواة نظام التشغيل (Kernel) الخاص بالجهاز المضيف.
4. كل حاوية هي حزمة تنفيذية مستقلة من البرمجيات تعمل فوق نظام التشغيل المضيف.
5. يمكن للجهاز المضيف تشغيل عدد كبير من الحاويات في الوقت نفسه.
6. تتميز الحاويات بأنها قابلة للنقل (Portable)، حيث يمكن تشغيلها على أي بنية تحتية تدعم تقنيات الحاويات مثل: Docker و Kubernetes.

مقارنة بين ال Virtualization وال Containerization:



واجه المقارنة	Containers	VMs
التعريف	تقوم بتغليف التطبيقات مع اعتمادياتها (dependencies) داخل حاويات تشترك في نواة نظام التشغيل المضيف	تقوم بتشغيل أنظمة تشغيل كاملة كـ "أنظمة ضيف" فوق جهاز فيزيائي باستخدام Hypervisor
طبقة التنفيذ Execution layer	يعمل محرك الحاويات (مثل Docker Engine) فوق نظام التشغيل المضيف	يعمل ال Hypervisor مباشرة على العتاد (Type 1), أو فوق نظام تشغيل مضيف (Type 2)
نظام التشغيل Operating system	تتشترك جميع الحاويات في نواة نظام التشغيل المضيف، ولا تحتاج إلى نظام تشغيل منفصل.	كل آلة افتراضية تحتاج إلى نظام تشغيل كامل خاص بها (linux أو windows)
العزل Isolation	عزل على مستوى العمليات باستخدام cgroups و namespaces ، وهو أخف لكنه أقل صرامة	عزل قوي على مستوى العتاد، ومناسب للبيئات عالية الأمان
استهلاك الموارد Resource Consumption	استهلاك منخفض، حيث تستهلك الموارد اللازمة للتطبيق فقط	استهلاك مرتفع للـ (ذاكرة، معالج، تخزين) بسبب تشغيل عدة أنظمة تشغيل
زمن الإقلاع Startup Time	سريع (ثوانٍ معدودة)	بطيء نسبياً (قد يستغرق دقائق)
الأداء Performance	خفيفة جداً وأداؤها قريب من الأداء الحقيقي مع حمل أقل	أداء قريب من الأداء الحقيقي ولكن مع بعض الحمل الزائد (Overhead)
قابلية النقل Portability	عالية جداً ويمكن نقلها بين البيئات المختلفة بسهولة (Docker, Kubernetes)	محدودة وتعتمد على نوع الـ Hypervisor ونظام التشغيل
الإدارة Management	إدارة أبسط، تحديثات أسرع، ودعم ممتاز لـ CI/CD.	إدارة معقدة نسبياً، وتتطلب تحديث وصيانة أنظمة تشغيل متعددة

الأمان Security	أمان جيد لكنه يعتمد على أمان نظام التشغيل المضيف ويحتاج إلى إعدادات حماية إضافية	أمان قوي بسبب العزل الكامل لكل آلة افتراضية
أفضل حالات الاستخدام	- تطبيقات Microservices - التطبيقات السحابية الحديثة (cloud-native apps) - DevOps pipelines - التوسع السريع rapid scaling	- الأنظمة المؤسسية - البيئات التي تتطلب التزاماً - عاليًا بالمعايير الأمنية، - وتشغيل أنظمة تشغيل متعددة
أمثلة	Docker Kubernetes Podman OpenShift	VMware ESXi Microsoft Hyper-V Oracle VM Citrix Hypervisor

فائدة اربيسيزية :

كيف توفر تقنية الحاويات الموارد؟

لنفترض أن لدينا جهاز حاسوب يحتوي على 16 جيجابايت من الذاكرة العشوائية (RAM) و 8 أنوية (Cores) للمعالج. في حالة الافتراضية التقليدية، قمنا باستخدام Oracle VM وأنشأنا ثلاث آلات افتراضية، يعمل على كل واحدة منها نظام تشغيل Ubuntu ، مع تخصيص الموارد على النحو التالي:

- 4 جيجابايت RAM و Core 2 لكل آلة افتراضية.

ثم قمنا بتشغيل:

- MySQL Server على الآلة الافتراضية الأولى،
- Redis على الآلة الافتراضية الثانية،
- Search Engine على الآلة الافتراضية الثالثة.

في هذا السيناريو، قد يصل استهلاك MySQL إلى ما يقارب 90٪ من الموارد المخصصة له، في حين لا يتجاوز استهلاك Redis 30٪. هنا تظهر مشكلة شائعة في الافتراضية :

الموارد تكون محجوزة مسبقاً لكل آلة افتراضية، ولا يمكن إعادة توزيعها ديناميكياً بين الخدمات، حتى لو كانت غير مستخدمة فعلياً. والنتيجة هي هدر في الموارد وأداء غير مثالي للخدمة التي تحتاج موارد أكبر.

أما عند تطبيق نفس المعمارية باستخدام الحاويات (Containerization) ، فإن الخدمات الثلاث تعمل مباشرة على نفس نظام التشغيل وتتشارك موارد الجهاز كاملة، أي 16 جيجابايت من الذاكرة و 8 أنوية المعالج. في هذه الحالة، إذا احتاجت خدمة قاعدة البيانات إلى 5 جيجابايت من الذاكرة، يمكنها استخدامها فعلياً، بينما تستهلك Redis فقط القدر الذي تحتاجه دون وجود موارد محجوزة وغير مستغلة.

هذا الأسلوب يؤدي إلى استغلال أفضل للموارد، وأداء أعلى، ومرونة أكبر في التعامل مع اختلاف أحمال الخدمات.

:Redis

قاعدة بيانات سريعة جداً تُخزن البيانات في الذاكرة (In-Memory)، تُستخدم غالباً للتخزين المؤقت (Caching).

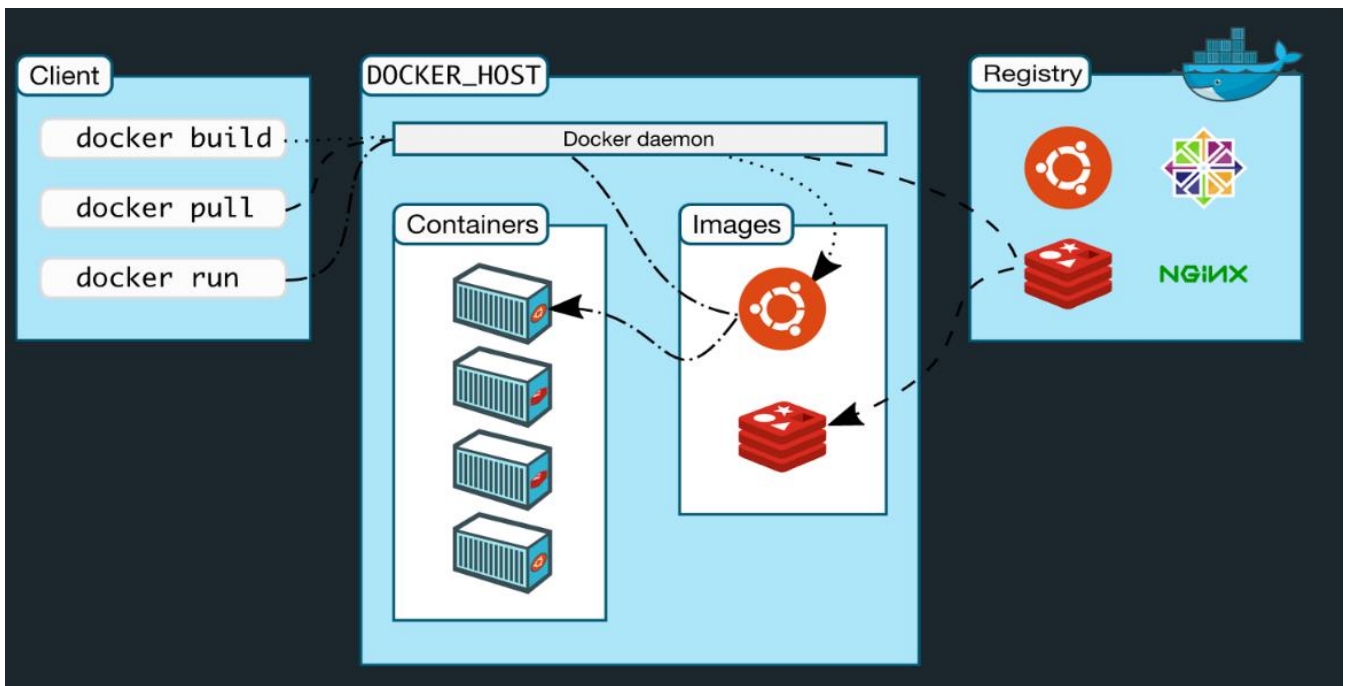
: Search Engine

نظام مخصص للبحث السريع داخل كميات كبيرة من البيانات، يعتمد على الفهرسة (Indexing) ويُستخدم لاسترجاع النتائج بدقة وسرعة مثل Elasticsearch

Docker

هو منصة حاويات برمجية تتيح لك بناء التطبيقات واختبارها ونشرها بسرعة، وذلك من خلال تغليف التطبيقات ضمن وحدات قياسية تسمى الحاويات (Containers).
تحتوي ال container على كل ما يحتاجه التطبيق للعمل، بما في ذلك المكتبات، وأدوات النظام، والكود البرمجي، وبيئة التشغيل (Runtime)، مما يضمن تشغيل التطبيق بشكل متناسق في مختلف البيئات.

بنية Docker (Docker Architecture)



1. عميل Docker (Docker Client)

- يُعد عميل Docker (الأمر Docker) الطريقة الأساسية التي يتفاعل من خلالها المستخدمون مع منصة Docker.
- عند تنفيذ أوامر مثل `docker run` أو `docker pull`، يقوم العميل بإرسال هذه الأوامر إلى خادم Docker (Docker Daemon) ليتم تنفيذها.
- يعتمد عميل Docker على Docker API للتواصل مع الخادم، كما يمكنه الاتصال بواحد أو أكثر من خوادم Docker لإدارة الحاويات والخدمات.

The best way to predict the future
is to create it.

2. مضيف Docker (Docker host)

يوفر مضيف Docker بيئة متكاملة لتشغيل التطبيقات المعتمدة على الحاويات، ويتكوّن من العناصر التالية:

- Docker Daemon
- الصور (Images)
- الحاويات (Containers)
- الشبكات (Networks)
- وحدات التخزين (Volumes)

3. خادم Docker (Docker Daemon)

هو العملية الأساسية التي تعمل في الخلفية ويتولى مسؤولية جميع العمليات المتعلقة بالحاويات، حيث يستقبل الأوامر عبر واجهة سطر الأوامر (CLI) أو من خلال REST API، كما يتولى إدارة كائنات Docker المختلفة مثل:

- الصور (Images)
- الحاويات (Containers)
- الشبكات (Networks)
- وحدات التخزين (Volumes)

كما يمكنه التواصل مع daemons أخرى لإدارة الخدمات وتنسيق العمل بينها.

4. سجل Docker (Docker Registry)

- سجل Docker هو مستودع تُخزّن فيه صور Docker
- يُعتبر Docker Hub السجل العام الافتراضي الذي يستخدمه Docker ، ويمكن لأي شخص الوصول إليه (open source)، كما يمكن للمؤسسات إنشاء سجل خاص (Private Registry) لاستخدامه داخلياً.
- عند استخدام الأمر `docker pull` أو `docker run`، يتم جلب الصور المطلوبة من السجل المُهيأ.
- عند استخدام الأمر `docker push`، يتم رفع الصور إلى السجل المحدد.

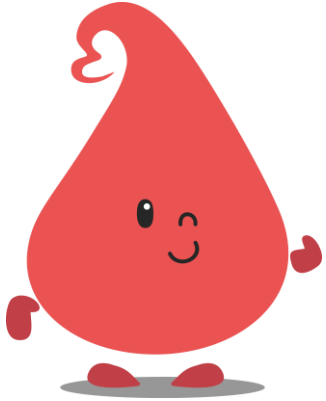
توضيح ارييسيزي :الفرق بين Private Registry و Private Image في Docker :

Private Registry

هو مستودع خاص لتخزين صور Docker بالكامل داخل المؤسسة أو الفريق. يتيح التحكم الكامل بمن يمكنه الوصول إلى جميع الصور داخل السجل. مثال: سجل داخلي يحتوي على كل صور تطبيقات الشركة ولا يمكن لأي شخص خارجي الوصول إليه.

Private Image

هي صورة Docker واحدة خاصة، يمكن رفعها على سجل عام أو خاص. الوصول إليها محدود للمستخدمين المصرح لهم فقط. مثال: صورة تطبيق أو قاعدة بيانات محمية، لا يمكن تحميلها إلا من قبل المطورين المخولين.

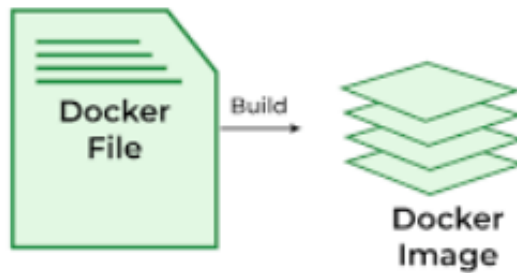


Images in Docker

الصورة (Image) هي قالب للقراءة فقط (Read-only) يحتوي على مجموعة من التعليمات اللازمة لإنشاء Docker Container, وغالبًا ما تكون الصورة مبنية على صورة أخرى مع بعض التخصيصات الإضافية.

مثلاً:

يمكن إنشاء صورة تعتمد على صورة Ubuntu, ثم يتم إضافة خادم الويب Apache والتطبيق الخاص بك, بالإضافة إلى ملفات الإعدادات اللازمة لتشغيل التطبيق.



يمكننا إنشاء صور خاصة بنا أو استخدام صور جاهزة (مثل MySQL) أنشأها مستخدمون آخرون وتم نشرها في سجل Docker (Registry).

ولبناء صورة خاصة بنا, نقوم بإنشاء ملف Dockerfile, يحتوي على تعليمات بسيطة تحدد الخطوات اللازمة لبناء الصورة وتشغيلها, ويجب أن تكون الإصدارات المحددة في الصورة المتوافقة مع التطبيق.

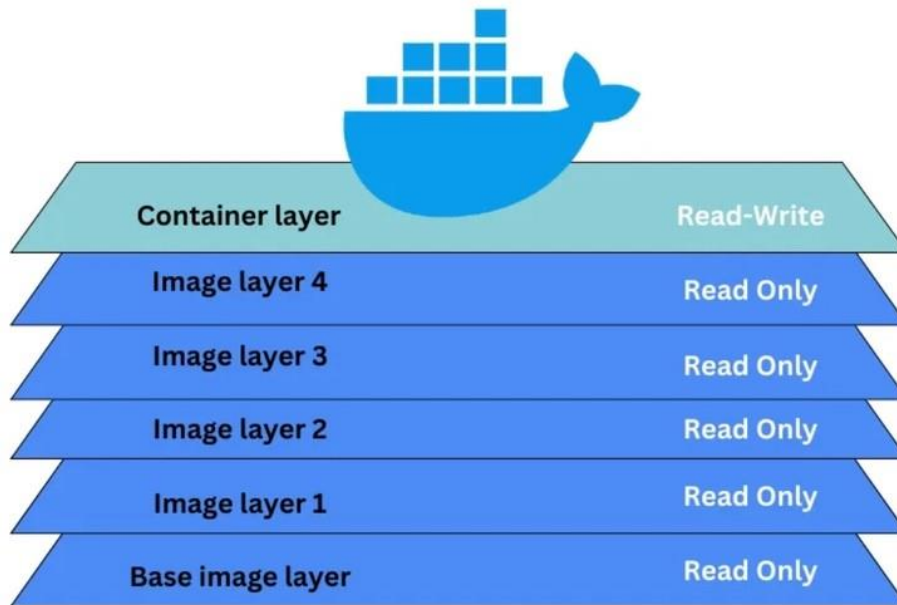
مثلاً:

عند إنشاء صورة لتطبيق Laravel, يجب تضمين جميع المتطلبات اللازمة لتشغيله, مثل:

- PHP بإصدار متوافق مع Laravel
- خادم ويب (مثل Apache أو Nginx)
- الامتدادات (Extensions) المطلوبة في PHP

ويجب التأكد من أن جميع هذه المكونات تعمل بإصدارات متوافقة فيما بينها, وإلا فقد يفشل التطبيق في العمل أو تظهر أخطاء أثناء التشغيل.





كل تعليمة داخل ملف Dockerfile تؤدي إلى إنشاء طبقة (Layer) جديدة ضمن الصورة. وعند تعديل Dockerfile وإعادة بناء الصورة، لا يتم إعادة بناء الصورة كاملة، وإنما تعاد فقط الطبقات التي طرأ عليها تغيير.

وهذه الآلية هي أحد الأسباب التي تجعل صور Docker خفيفة الحجم، صغيرة، وسريعة مقارنة بتقنيات الافتراضية التقليدية.

Containers in Docker

الحاوية (Container) هي نسخة قابلة للتشغيل (Runnable Instance) من الصورة. يمكننا باستخدام Docker API أو واجهة سطر الأوامر (CLI) إنشاء، تشغيل، إيقاف، نقل وحذف ال Container كما يمكن:

- ربط الحاوية بشبكة واحدة أو أكثر
- إرفاق وحدات تخزين بها
- أو حتى إنشاء صورة جديدة اعتماداً على الحالة الحالية لل container

بشكل افتراضي، تكون الحاوية معزولة نسبياً عن باقي الحاويات وعن النظام المضيف. ويمكن التحكم بدرجة هذا العزل فيما يخص: الشبكة، التخزين أو الأنظمة الفرعية الأخرى، سواء بالنسبة للحاويات الأخرى أو بالنسبة للنظام المضيف.

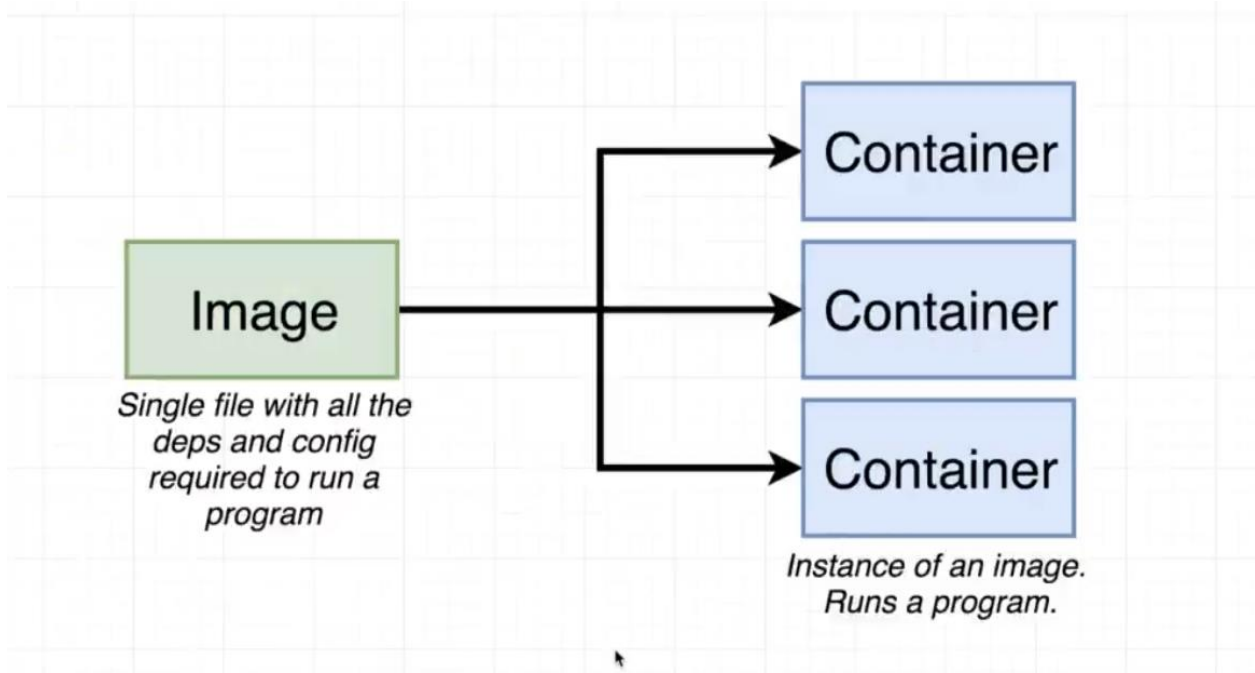
يتم تعريف الحاوية بواسطة:

- الصورة التي تعتمد عليها
- وخيارات الإعداد التي يتم تحديدها عند إنشائها أو تشغيلها

عند تشغيل حاوية اعتمادًا على Image محددة:

- يتحقق Docker أولاً مما إذا كانت الصورة موجودة محلياً.
- إذا لم تكن موجودة، يتم البحث عنها في Docker Registry (مثل Docker Hub)
- يتم تحميل الصورة تلقائياً باستخدام docker pull

عندما نقوم بتحديث image معينة، جميع ال containers التي تعتمد على هذه الصورة تتحدث تلقائياً وعند حذف الحاوية، فإن أي تغييرات طرأت على حالتها تختفي ما لم تكن محفوظة في وحدة تخزين دائمة (Persistent Storage)



الخاصية	Container	Images
التعريف	نسخة قيد التشغيل من الصورة، تمثل التطبيق فعلياً على المضيف	نسخة ثابتة جاهزة من التطبيق مع جميع اعتماديّاته (كود، مكتبات، أدوات النظام، بيئة التشغيل)
الحالة	ديناميكية وقابلة للتعديل أثناء التشغيل	ثابتة ولا تتغير
الاستخدام	تُستخدم لتنفيذ التطبيق وتشغيله	تُستخدم لإنشاء حاويات
التغيير	يمكن تعديل الملفات والعمليات داخل الحاوية أثناء التشغيل	لا يمكن تعديل الصورة مباشرة، يجب إنشاء نسخة جديدة مع التعديلات
أمثلة	الحاويات التي تشغل Laravel App على الخادم	صورة Laravel App جاهزة مع PHP و Composer

■ التخزين في Docker (Storage)

يمكن تخزين البيانات داخل الطبقة القابلة للكتابة (Writable Layer) الخاصة بالحاوية، ولكن هذا النوع من التخزين:

- غير دائم (Non-persistent)
 - يتم فقدانه عند إيقاف أو حذف الحاوية.
 - يصعب نقله أو مشاركته مع حاويات أخرى.
- لذلك لا يُنصح بالاعتماد عليه لتخزين البيانات المهمة، ولهذا السبب يوفر Docker عدة آليات للتخزين الدائم وهي :

1. Docker Volumes

- تُعد Docker Volumes أكثر طرق التخزين الدائم استخداماً في Docker
- يتم إدارتها بالكامل من قبل Docker حيث تمتلك نظام ملفات مخصصاً على الجهاز المضيف ولا تعتمد على بنية نظام الملفات في هذا النظام المضيف.
- يتم التحكم بال Docker Volumes عن طريق Docker command line
- يمكن إنشاء الـ Volume بشكل مستقل، أو أثناء إنشاء الحاوية.
- عند إيقاف أو حذف الحاوية، تبقى الـ Volumes محفوظة ولا تُحذف.
- يتم تخزين بيانات Docker Volumes افتراضياً في المسار التالي على النظام المضيف (في Linux):
/var/lib/docker/volumes

أما في Windows وماك تدار بواسطة docker engine

آلية عمل Volume عند تشغيل الحاوية

- عند تشغيل حاوية مع تحديد Volume يتحقق Docker أولاً مما إذا كان الـ Volume موجوداً. إذا كان موجوداً، يتم استخدامه مباشرة، أما إذا لم يكن موجوداً يقوم Docker بإنشائه تلقائياً.
- عند إعادة تشغيل الحاوية، يتم إعادة ربط نفس الـ Volume بحيث لا تُفقد البيانات.

2. Docker Bind Mount

- يُعد خياراً ثانياً للتخزين الدائم، لكنه:
- يعتمد كلياً على نظام الملفات في الجهاز المضيف.
- لا يتم إدارته بشكل كامل من خلال Docker CLI مثل Volumes .
- يمكن إنشاء أو تحديد مسار من نظام ملفات المضيف أثناء تشغيل الحاوية، ويتم ربطه مباشرة داخل الحاوية.

Docker Volumes أسرع من Bind Mounts لأنها:

تُدار مباشرة من قبل Docker مما يسمح له بالتحكم الكامل في آلية الربط والتخزين دون الاعتماد على بنية نظام الملفات في الجهاز المضيف.

تقلل عدد طبقات القراءة والكتابة (I/O Layers)

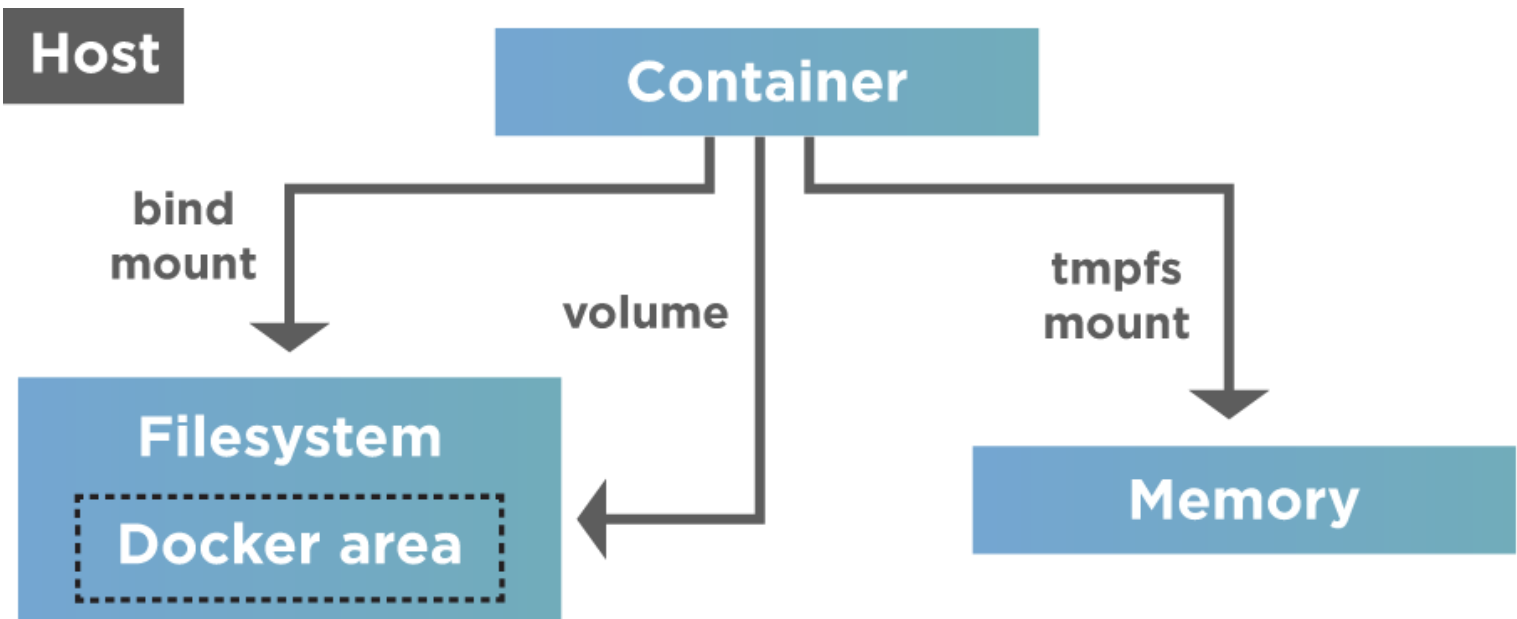
حيث تتم عمليات الإدخال والإخراج عبر مسار أقصر مقارنةً بـ Bind Mounts التي تمر عبر نظام ملفات المضيف.

3. Tmpfs

- خيار تخزين غير دائم، حيث يتم تخزين البيانات مباشرة في ذاكرة RAM الخاصة بالجهاز المضيف (أو swap عند الامتلاء)، ويتم حذف البيانات فور إيقاف الحاوية.
- مفيد جدًا عند التعامل مع بيانات مؤقتة لا يُراد تخزينها بشكل دائم.
- لا يمكن مشاركة tmpfs بين الحاويات إلا على أنظمة Linux .
- عند استخدام tmpfs ، يمكن الاعتماد على:
 - --tmpfs
 - أو الخيار الأحدث --mount الذي يوفر إعدادات أكثر مرونة.
- لا يُنصح باستخدام tmpfs للبيانات المهمة أو في الأنظمة التي تتطلب الاستمرارية، لأنه يعتمد على الذاكرة ويؤدي إلى فقدان البيانات عند إيقاف الحاوية أو إعادة تشغيل النظام.

4. Storage Plugins

- تتيح ربط Docker بمنصات تخزين خارجية.
- يتم من خلالها ربط التخزين من الجهاز المضيف بمصادر خارجية مثل:
 - Storage Arrays
 - Appliances
- يمكن الاطلاع على الإضافات المتاحة من صفحة Docker الرسمية للإضافات (Plugins) .



What we think, we become

■ الشبكات في Docker (Networking)

تمثل شبكات Docker القناة التي تتواصل من خلالها الحاويات المعزولة مع بعضها البعض ومع العالم الخارجي. يوفر Docker عدة برامج تشغيل للشبكات (Network Drivers) ، ومن أهمها:

1. Bridge Network

- هو برنامج التشغيل الافتراضي للحاويات في Docker
- يُستخدم عندما تعمل الحاويات بشكل مستقل (Standalone Containers) على نفس الجهاز المضيف.
- يسمح بتواصل عدة حاويات مع بعضها عبر نفس Docker Host
- مناسب لتطبيقات تعمل على خادم واحد.



2. Host Network

- يقوم هذا النوع بإلغاء العزل الشبكي بين الحاوية والجهاز المضيف.
- تشترك الحاوية مع المضيف في نفس واجهة الشبكة.
- يُستخدم عندما لا تكون هناك حاجة لعزل شبكي.

3. Overlay Network

- يتيح هذا النوع من الشبكات التواصل بين الحاويات التي تعمل على أجهزة مضيئة مختلفة.
- يُستخدم بشكل أساسي مع Docker Swarm لربط الخدمات (Services) ببعضها.
- مناسب للتطبيقات الموزعة والأنظمة متعددة الخوادم.

4. Macvlan Network

- يقوم هذا النوع بإسناد عنوان MAC خاص لكل حاوية، مما يجعلها تبدو كأنها جهاز فعلي على الشبكة.
- يتم توجيه حركة البيانات بين الحاويات اعتماداً على عناوين MAC الخاصة بها.
- يُستخدم في الحالات التي يُراد فيها أن تتعامل الشبكة مع الحاويات كأجهزة فيزيائية حقيقية، مثل حالات ترحيل الأنظمة الافتراضية (VM Migration)

خلاصة :

- Bridge : يُستخدم لربط الحاويات التي تعمل على نفس الجهاز المضيف.
- Host : تعمل الحاوية بدون عزل شبكي وتشارك المضيف واجهة الشبكة مباشرة.
- Overlay : يُستخدم لربط الحاويات التي تعمل على عدة أجهزة مضيئة ضمن بيئة Docker Swarm.

Macvlan: تجعل الحاوية تظهر على الشبكة وكأنها جهاز فيزيائي مستقل بعنوان MAC خاص بها.

The End